TI-86 Calculator Link Relay

ATOM Lite hardware build and first safe test

2026-08-02

Read this first

This guide connects a TI-86 calculator's 2.5 mm link cable to an M5Stack ATOM Lite. The ATOM becomes a relay that can later move catalog data, lesson packages, and quiz results between the calculator and DaylightStation.

The calculator cable does **not** plug directly into the ATOM Lite. Red and white are 5-volt calculator signal wires. A very small protection circuit must sit between them and the ATOM Lite.

The first firmware test takes a **read-only screenshot**. It does not write, delete, or modify calculator data. Do not enable that test until the meter checks in this guide pass.

What you are building

```
TI-86 2.5 mm cable                  M5Stack ATOM Lite

red   ---+
white ---+--- small protection board --- four GPIO wires --- ATOM Lite
black ---+              |
                       +--- common ground
```

The small board makes the calculator's 5 V signals safe for the ATOM's 3.3 V inputs, and lets the ATOM pull a calculator line low when communicating without ever forcing a line high.

What to buy

M5Stack parts

- **M5Stack ATOM Lite**, SKU **C008** — you already have this controller.
- **M5Stack ATOM Hub Proto Kit**, SKU **K039** — recommended. It is an off-the-shelf M5Stack case and solderable prototype board for the ATOM.

The Hub Proto is not a complete TI calculator adapter. It gives this small circuit a proper home instead of leaving it loose on wires.

Do **not** buy an ATOMIC RS-232 Base for this project. RS-232 is a different electrical language from the calculator link.

Small electronic parts

Quantity	Part	Job
2	2N7000, through-hole N-channel MOSFET	safely pulls one calculator line low
2	10 kΩ resistor	first half of each input safety divider

Quantity	Part	Job
2	15 kΩ resistor	second half of each input safety divider
4	100 Ω resistor	protects two ATOM input leads and two MOSFET gate leads
2	100 kΩ resistor	keeps each MOSFET off while the ATOM boots
1	2.5 mm TRS breakout/socket	only if your cable does not already end in bare wires
1	hookup wire, heat-shrink, and cable tie	connections and strain relief
1	digital multimeter	required for the safety checks

If your cable already has bare red, white, and black conductors, route those three wires into the Hub Proto board directly. You do not also need a TRS breakout.

Do not substitute these blindly

- Do not use a USB-to-TI Graph Link cable. The standard ATOM Lite is not a USB host for that cable.
- Do not use the M5Stack RS-232 base.
- Do not connect red or white directly to any ATOM GPIO.
- Do not use a generic four-channel I^2C level-shifter board as a drop-in replacement. Those boards expect their own powered pull-up rails and do not automatically provide the calculator-safe boot behavior required here.

The four ATOM Lite connections

Purpose	ATOM Lite GPIO	Where it is exposed
Listen to calculator red/tip line	GPIO32	Grove/CUSTOM port
Safely pull calculator red/tip line low	GPIO25	ATOM pin interface / Hub Proto breakout
Listen to calculator white/ring line	GPIO33	ATOM pin interface / Hub Proto breakout
Safely pull calculator white/ring line low	GPIO26	Grove/CUSTOM port
Common ground	GND	Grove/CUSTOM port or Hub Proto ground

The firmware already uses this map. Do not change it until the first test works. The ATOM's 5 V pin is **not used** in the calculator circuit.

The three calculator wires

Cable color	2.5 mm plug contact	Meaning
red	tip	calculator link line A
white	ring	calculator link line B
black	sleeve	ground

Check this with the continuity setting on your meter. Put one probe on the **tip** and touch the bare conductors until it beeps. Repeat for the ring and sleeve. Do not trust colors alone.

Connect black/sleeve directly to ATOM **GND**. Red/tip and white/ring each use one copy of the circuit on the next page.

Build the protection circuit

Build the red/tip channel first. Then build the identical white/ring channel.

```
RED / TIP calculator wire
        |
        +---------------- to MOSFET DRAIN
        |
        +--- 10 kOhm ---+--- 100 Ohm --- ATOM GPIO32  (listen to red)
                         |
                       15 kOhm
                         |
BLACK / GROUND ----------+---------------- ATOM GND

ATOM GPIO25 --- 100 Ohm --- MOSFET GATE
                              |
                            100 kOhm
                              |
GROUND ---------------------+

MOSFET SOURCE --------------- GROUND
```

For the white/ring channel, make the same circuit but replace:

```
RED/TIP  => WHITE/RING
GPIO32   => GPIO33
GPIO25   => GPIO26
```

The 10 kΩ and 15 kΩ pair make a safe listening connection: a 5 V calculator signal becomes about 3.0 V at the ATOM input. The MOSFET is the talking connection: it can connect a calculator line to ground, but cannot force it high. The 100 kΩ resistor keeps that switch off during boot and reset.

MOSFET warning

Use the data sheet for the exact 2N7000 part you buy to identify its Gate, Drain, and Source legs. Do not rely on a random web image; physical lead order can differ between parts or manufacturers.

Build order

1. Mount the ATOM Lite in the ATOM Hub Proto case, but leave the calculator cable disconnected.
2. Solder the black calculator wire to the circuit-board ground rail.
3. Run one ground wire from that rail to ATOM GND.
4. Build the red channel exactly as shown above.
5. Build the white channel exactly as shown above.
6. Add heat-shrink or a cable tie so pulling the calculator cable cannot pull on solder joints.
7. Inspect every joint with a magnifier before applying power.

Safety checks with a meter

Do these checks **before plugging the cable into the calculator**.

1. No shorts

With the ATOM unpowered, check that black-to-red, black-to-white, and red-to-white do **not** beep as a direct short. A momentary changing resistance is normal; a constant beep or near-zero resistance is not.

2. MOSFET gates are off

Power the ATOM by USB, but leave the calculator cable unplugged. Flash the firmware with `transmit_enabled: false` (the example configuration already does this). Measure each MOSFET gate relative to ground:

- it should be near 0 V;
- it must not be near 3.3 V.

This proves the 100 kΩ safety resistors are keeping both switches off.

3. Safe idle firmware

Still with no calculator connected, visit:

```
http://<ATOM-IP>/health
```

Confirm this is present:

```
"ti_link": { "transmit_enabled": false }
```

At this setting the firmware can observe input levels but cannot turn on either MOSFET.

4. Calculator attached, still observe-only

With `transmit_enabled: false`, plug in the TI calculator. Nothing should happen: no menus, no error, no reset, and no apparent key press. If anything unusual happens, unplug immediately and inspect the board.

First real test: read a screenshot

Only proceed after every safety check passes.

1. Set this in the private `ticalc-relay.yml` configuration:

   ```yaml
   link:
     transmit_enabled: true
     foreground_listener: false
     auto_sync: false
   ```

   Keep both automatic paths off for this first read-only diagnostic. The foreground listener is enabled only after the screenshot and variable tests prove the protected interface.

2. Regenerate configuration and flash the ATOM:

   ```
   cd _extensions/ticalc-relay/firmware
   node tools/gen-config.mjs <path-to>/ticalc-relay.yml <relay-id>
   pio run -e m5-atom -t upload --upload-port /dev/cu.usbserial-XXXX
   ```

3. Turn on the TI-86 and leave it at a normal screen where it can accept a key.

4. From a computer on the same network, request the local diagnostic:

   ```
   curl -X POST http://<ATOM-IP>/diagnostics/link/screenshot
   ```

   A successful queue response is:

   ```
   {"ok":true,"state":"screenshot_queued"}
   ```

5. Wait a few seconds, then inspect:

   ```
   curl http://<ATOM-IP>/status
   ```

   Success means `screenshot_ready: true`, `screenshot_count: 1`, and `last_error: "none"` under `ti_link`.

6. Save the raw 128×64 monochrome frame for later comparison:

   ```
   curl http://<ATOM-IP>/diagnostics/link/screenshot.raw > ti86-screen.raw
   ```

The documented read-only transaction is:

```
ATOM -> TI-86: 06 6D 00 00                ask for a screenshot
TI-86 -> ATOM: 86 56 00 00                acknowledge
TI-86 -> ATOM: 86 15 00 04 + 1024 bytes   send screen bitmap
ATOM -> TI-86: 06 56 00 00                acknowledge receipt
```

The screenshot is 1,024 bytes: 128×64 pixels, one bit per pixel. This is the first test because it reads from the calculator but does not write calculator variables.

If the first test fails

Status `last_error`	Likely cause	Safe next action
`TI lines were not both idle/high before transmit`	short, wrong MOSFET legs, or a line held low	unplug calculator; repeat no-short and gate-voltage checks
`timed out waiting for TI link handshake edge`	wrong tip/ring wiring, bad cable, calculator not ready, or a wrong GPIO wire	turn transmit off; verify continuity and all four GPIO wires
`invalid TI link edge`	both lines are pulled low together	inspect red/white channels for a solder bridge or reversed MOSFET
`TI packet data checksum failed`	electrical noise or marginal wiring	shorten wires, inspect ground, retry once; do not try variable writes
`unexpected TI packet during screenshot transaction`	link is alive but a protocol assumption needs inspection	save `/status`; do not add variable-write code yet

If a failure persists, return `transmit_enabled` to `false` before changing hardware. The MOSFET gate pulldowns keep both calculator lines released when the ATOM is reset or unpowered.

What the test proves

After a successful screenshot, we know the cable, ground, dividers, MOSFET switches, ATOM pins, timing task, and basic TI packet framing work together. It does **not** yet prove variable transfer, lesson installation, catalog sync, or result upload. The next increment is one harmless test data variable, including calculator acknowledgement, readback, and interrupted-retry testing.

After variable transfer: test calculator-initiated SchoolCalc sync

Only after the screenshot and harmless-variable matrix passes, set:

```yaml
link:
  transmit_enabled: true
  foreground_listener: true
  auto_sync: false
```

Regenerate, flash, and leave the relay idle. `/status` must show `foreground_listener_state: "armed_unknown_idle"` only when the interface sees a usable idle bus; this is readiness, not a verified connection. On the TI-86, open SchoolCalc and press Sync. No LAN request and no `LINK > RECV` action should be needed. The expected sequence is:

1. relay status changes to `hello_candidate`/`negotiating` and LED blue;
2. the nonce-matched handshake changes connection evidence to `verified_session`;
3. direction changes cyan → yellow → purple as calculator upload, backend work, and calculator download occur;
4. the calculator and relay both end with either a named safe error or terminal success, never a perpetual busy state; and
5. `/status` records `last_initiator: "calculator"` and increments `calculator_initiated_sync_count`.

Repeat with the cable pulled during HELLO, each direction, and final staging. Each attempt must release both sinks, say safe to unplug after stopping, preserve the old committed content and `DSQ`, and remain retryable. Keep `auto_sync: false`; that separate flag is the relay-initiated Silent Link poll.

Sources

- M5Stack, ATOM Lite documentation and pin map, accessed 2026-08-01. It identifies GPIO26/GPIO32 on the custom port and GPIO25/GPIO33 on the pin interface.
- M5Stack, ATOM Hub Proto Kit, accessed 2026-08-01. It documents the K039 proto board/case as an ATOM expansion platform.
- TI-86 Link Protocol Guide, link cable and two-wire handshake, accessed 2026-08-01.
- TI-86 Link Protocol Guide, screenshot packet sequence, accessed 2026-08-01.